Most of these functions should make sense just from reading their names and understanding PHP's data typing system. Let's have a couple of examples before I explain why `empty()` should be avoided:

```
var_dump(is_array([])); // bool(true)
var_dump(is_array(new ArrayObject())); // bool(false)
var_dump(is_iterable(new ArrayObject())); // bool(true)
var_dump(is_numeric(1)); // bool(true)
var_dump(is_numeric(1.1)); // bool(true)
var_dump(is_numeric('1')); // bool(true)
var_dump(isset($notDeclared)); // bool(false) - no error output
```

`empty()` Is a Problem

Most of these functions are quite specific in their requirements: the input needs to match strict type or value requirements, which makes them predictable and therefore safe. The function names themselves are self-explanatory: skimming through code that contains these functions should make sense without requiring too much thought. And then we have `empty()`. This function is a bit of a Swiss Army Knife: it is the equivalent of calling `isset()`, testing for values such as `false`, `null`, and empty `[]` – it does rather a lot of different things depending on context. The following demonstrates all the various ways that `empty()` can return true:

```
var_dump(empty($notDeclared)); // bool(true)
var_dump(empty(false)); // bool(true)
var_dump(empty(0)); // bool(true)
var_dump(empty(0.0)); // bool(true)
var_dump(empty(''')); // bool(true)
var_dump(empty('0')); // bool(true)
var_dump(empty([])); // bool(true)
```

That's quite a variety of different states that `empty()` will happily report as true. You're probably better off using more specific tests such as `isset()` for non-declared variables, `=== 0` for integer zero, `strlen() === 0` for empty string, and so on. `empty()` on an empty array makes the most sense semantically, so it's not without its uses, but tread carefully around code that contains `empty()` in logical expressions. Repeat after me: type-safe code is safer code.